1 Introduction

1.1 Background and Motivation

Code review involves examining source code or proposed changes to identify problems before those changes enter a shared codebase. Modern code review combines the examination of code changes with communication between developers. A case study at Google identifies code understandability, knowledge sharing, and consistency as important review goals alongside defect detection [1].

Reading techniques differ in how they guide the reviewer. Ad hoc review leaves the choice of focus and procedure to the reviewer, whereas checklist-based review provides predefined questions or checks [2]. More structured techniques provide inspection procedures, including scenarios associated with particular stakeholder perspectives [3].

A checklist provides one way to make review criteria explicit. A checklist item directs attention to a property that should be examined, such as whether implementation behaviour agrees with a documented application programming interface (API) contract or whether tests cover a relevant boundary case. Checklist construction requires more than a list of broad quality goals. Chong et al. analysed 1,791 student-authored checklist questions and classified 425 as inappropriate, including questions that were ambiguous or unrelated to the assigned task [4]. A question such as “Is the code maintainable?” does not specify which concrete property the reviewer should examine [4, p. 27].

Research on human reviewers provides reasons to investigate explicit criteria, but it also warns against assuming that a checklist is inherently effective. Gonçalves et al. find task-dependent results when comparing unstructured review, a conventional checklist, and a guided checklist [2]. Braz et al. find that an explicit security focus improves vulnerability detection, while the tested checklists provide no statistically significant additional benefit over that instruction [5]. These findings concern human reviewers.

Large language models (LLMs) support code review by analysing proposed changes and generating comments for developers. AutoCommenter checks coding practices at Google [6], while RovoDev generates and filters review comments within an industrial pull-request workflow [7]. More interactive approaches allow an automated reviewing agent to inspect repository files and execute checks before reaching a decision. SWE-Review uses such an agentic process to assess whether a candidate patch resolves an issue and to provide feedback for revision [8].

In this thesis, a checklist is a structured set of review checks, whereas a prompt is the complete instruction supplied to a model. A prompt may contain a checklist alongside a task description, repository context, tool instructions, and output requirements. This working distinction allows the study to examine checklist content while controlling other instructions. The distinction does not imply that a checklist embedded in a prompt can have an effect independent of that prompt.

This thesis focuses on the criteria that guide an automated reviewer and how those criteria are applied during review. Existing LLM-based reviewers use several forms of explicit review guidance: grouped guidelines in RovoDev [7], specialised prompt experts in iCodeReviewer [9], and a fixed checklist in LLM-Reviewer [10]. Their use of explicit guidance motivates further investigation of which checks to select and how to execute them for backend API review.

1.2 Problem Statement and Research Gap

This thesis addresses the use of thematic checklists in agentic review of backend API changes. The scope includes implementation, tests, and documentation. For example, a change to an endpoint may require examination of its behaviour, the tests that exercise that behaviour, and the description exposed to API users.

Checklist content. RovoDev provides separate guideline groups for code review, test-file review, and review-comment construction. The authors removed these groups together and observed a five-percentage-point decrease in alignment with the locations of human review comments [7]. This result shows the contribution of the combined guidelines to that measure, but does not identify which individual checks account for it.

Checklist execution and context. LLM-Reviewer includes its checklist in the prompt used to analyse code [10]. iCodeReviewer uses specialised prompt chains, decomposes complex security checks into smaller propositions, and retrieves context relevant to particular checks [9]. These approaches illustrate different ways of organising model-guided review. However, their evaluations do not directly compare whole-checklist and stepwise execution while keeping the checklist content, model, and review cases unchanged.

1.3 Aim and Research Questions

The aim of this thesis is to develop and evaluate an agentic code-review system that automatically selects and applies thematic checklists to backend API changes.

The study addresses the following research question:

RQ: How do the content and execution of thematic checklists affect review quality and computational cost in agentic review of backend API changes?

Two subquestions guide the evaluation:

1. **SQ1 — Checklist content:** How does the use of thematic checklists affect review quality and cost, and how do those outcomes change when selected items or groups of items are removed?
2. **SQ2 — Checklist execution:** How do whole-checklist and stepwise execution, and shared and check-specific context, affect the quality and cost of agentic review?

1.4 Proposed Approach and Evaluation Overview

System workflow. The system will receive a pull request (PR), its task description, and repository context. It will analyse the change and select applicable checklists for implementation, tests, and documentation from the manually prepared collection. It will then use repository information and available tools to perform the selected checks and produce findings supported by code, requirements, or project rules.

Evaluation. The study will compare system variants on the same review cases to address the research question and its two subquestions. Each comparison will change the checklist content, execution strategy, or context strategy while keeping the model and other relevant conditions unchanged.

The evaluation cases will be drawn from suitable open datasets and supplemented with specially prepared changes where necessary. They will cover backend API implementation, tests, and documentation. Reference information about known problems and their supporting evidence will help assess the generated findings. Review quality will be assessed through the correctness and usefulness of findings, alongside measurements of system execution time and token consumption.

1.5 Expected Contribution

The expected contribution comprises a functioning agentic review system and empirical evidence about thematic checklist use in backend API review. The system will provide automatic selection of relevant checks for implementation, tests, and documentation. The experimental results will provide evidence about how the tested checklist content and execution strategies affect review quality, token consumption, and execution time.

2 Literature Review

2.1 Scope and Organisation of the Review

This chapter presents a thematic review of checklist design, human review, and LLM-based review systems, with particular attention to execution, context, and evaluation. It synthesises the selected literature rather than reporting an exhaustive systematic search.

The comparison distinguishes controlled findings from deployment observations and design proposals. Practitioner guidance informs checklist construction and process design, while empirical studies provide evidence about outcomes within their tested tasks and populations.

2.2 Code Review as a Technical and Collaborative Process

2.2.1 From Formal Inspection to Modern Review

Fagan described formal inspection as a structured review method with defined roles and stages, including preparation, defect identification, correction, and verification of the corrections [11].

Research following Fagan explored changes to both the inspection process and its supporting tools. Aurum et al. reviewed two-person inspections, inspections without a group meeting, and tool support for distributed and asynchronous inspection [12, Secs. 4.2, 4.5, and 5.2].

Modern code review uses a less formal, tool-supported workflow centred on individual code changes [1]. Reviewers inspect a proposed change, exchange comments with the author, and examine revised versions asynchronously.

2.2.2 Knowledge Sharing and Automated Assistance

Code review supports knowledge sharing as well as defect detection. Sadowski et al. identify knowledge transfer as an important goal of review at Google [1]. Work on automated review assistants also considers how developers interpret and discuss the feedback they receive [10], [13].

In the educational deployment of LLM-Reviewer, the checklist-based prompt is designed to provide recommendations without ready-to-use code solutions [10]. Students remain responsible for interpreting the feedback and implementing changes.

Tufano proposes conversational assistants that ask context-aware questions, respond to developer explanations, and help reviewers discuss trade-offs [13]. Rao et al. propose tailoring retrieved information and summaries to the reviewer's background and ordering related changes according to familiarity [14]. They highlight the difficulty of filtering irrelevant information without omitting evidence that could help the reviewer. Both papers present research proposals without experimentally evaluating the proposed assistants.

2.2.3 Reading Strategies and Explicit Criteria

Reading strategies differ in how much guidance they provide. Ad hoc review leaves the reviewer to choose an order and focus; checklist-based review supplies explicit questions; scenario-based techniques specify how to proceed [2], [3]. These approaches can overlap: Perspective-Based Reading uses scenarios associated with stakeholder perspectives, while a guided checklist combines questions with an execution procedure.

Basili et al. studied Perspective-Based Reading for requirements documents, using perspectives such as designer, tester, and user [3]. Each perspective has a goal and an inspection procedure.

2.3 Checklist Design and Evidence from Human Review

2.3.1 Specificity, Scope, and Common Authoring Errors

Drawing on recommendations from earlier literature, Gonçalves et al. designed their checklist so that items required specific answers, were separated by topic, and specified their scope, such as a method or class [2]. These properties clarify what the reviewer should inspect and where each question applies.

The *Inspection Manual* describes checklist questions as interpretations of specified rules and recommends focusing on major defects [15, p. 16].

Chong et al. analysed 1,791 checklist questions written by 394 students and classified 425 as inappropriate [4]. They identified four common mistakes: questions that were unclear, irrelevant, dependent on testing, or dependent on static analysis. The authors also noted an emphasis on documentation defects in student checklists. They recommended that educators give students more guidance on structural defects to better reflect the industrial review findings cited in the paper [4, Sec. VI].

2.3.2 Checklist Presence Does Not Guarantee Better Detection

Braz et al. separated the effect of directing attention from the effect of supplying a checklist [5]. Their online experiment involved 150 participants reviewing one prepared Java change with two seeded vulnerabilities. The baseline condition used ordinary review. Three other conditions provided a security-focus instruction, either alone or with a broader 22-item security checklist or a tailored seven-item security checklist. Under security-focused conditions, participants had at least eight times the odds of detecting a vulnerability as participants in ordinary review.

Neither checklist produced a statistically significant improvement over the security instruction alone [5]. The narrow task limits generalisation, as does the smaller participant subset in the checklist comparison. The design nevertheless demonstrates the value of an instruction-only baseline: it separates the benefit of directing attention from the additional benefit of a checklist.

Some participants cited insufficient knowledge as a reason why the checklist did not help [5]. The authors describe checklists as reminders that cannot substitute for the knowledge and analysis needed to judge a change [5].

2.3.3 Guidance, Task Complexity, and Cognitive Load

Gonçalves et al. compared ad hoc review, a conventional checklist, and a guided checklist in an experiment with 67 professional developers [2]. Both checklist conditions used the same content. The guided interface segmented the task, highlighted relevant code, and presented focused questions for particular code portions. The experiment therefore distinguishes checklist content from the way the checklist is delivered.

The conventional checklist showed an advantage for the most complex change, while the guided condition provided initial evidence of a benefit for a smaller task. Guided review showed no robust general advantage across all three tasks [2]. Low overall detection, limited prior review experience, and poor usability ratings constrain interpretation.

In some analyses, greater cognitive load accompanied better detection, which the authors interpret as potentially reflecting task engagement [2]. This association supports assessing effort together with outcomes, but does not show that increasing load causes better review.

2.4 Review Criteria in Automated Code Review

Code-review automation includes change analysis, reviewer recommendation, comment generation, and revision support [16]. The following discussion focuses on how review criteria are represented and where they originate.

LLM-Reviewer implements a checklist through a GitHub Action that submits changed code with a fixed 14-check prompt [10]. The checks span eight categories and include, for example, naming quality, duplicate code, and input validity. The prompt supplies all checks together and requests feedback by category. The report does not describe applicability-based selection or experimentally isolate the contribution of the checklist.

RovoDev also supplies explicit instructions, organised into code, test-file, and comment-construction guidelines, together with the PR description, linked task information, and code changes [7]. Code and test guidelines

concern assessment of the change; comment guidelines concern feedback construction. The latter are not checks of project documentation. The published pipeline supplies all three groups together rather than routing among thematic checklists. The ablation study removes all three guideline groups as one component and measures both location-only and location-and-semantic alignment with human comments [7]. It supports the contribution of the combined guidance but leaves individual code and test checks unevaluated.

AutoCommenter applies coding practices through learned predictions [6]. Its training examples associate human comments with locations and links to practice documents. The model predicts locations and practice references. Unlike LLM-Reviewer, it does not execute a manually supplied checklist.

Review history also provides material for manually written criteria. Practitioner guidance recommends adding checks for important recurring defects and revising personal checklists as habitual errors change [17], [18]. Aggarwal and Ghalaty describe engineers manually promoting accepted comments into version-controlled behavioural rules and a self-review checklist for coding agents [19]. Engineers assess whether lessons generalise and review the rule changes.

2.5 Execution and Context in Automated Review

2.5.1 Execution Strategies and Repository Context

iCodeReviewer selects specialised security checks based on code features and executes them through prompt chains [9]. For complex conditions, the LLM evaluates individual propositions, and the system combines the answers into a final decision. Some chains include intermediate analysis prompts. F1 decreased when the authors replaced the prompt experts with direct questions; the decrease was smaller when they removed analysis prompts [9]. However, the direct-question comparison changes both the domain-specific guidance and the execution structure, so it does not isolate the effect of stepwise execution.

The execution of checks also depends on access to relevant context. In the educational deployment of LLM-Reviewer, participants reported missing system context and limitations in reasoning across files [10]. For context-sensitive analyses, iCodeReviewer retrieves related code elements when required [9]. Rao et al. propose an assistant that selects and summarises repository information and analysis results, but do not experimentally evaluate this design [14].

SWE-Review lets an agent search a repository, inspect dependencies, and run commands before approving a candidate patch or requesting changes with a diagnosis [8]. The reviewer does not receive the reference patch or hidden test results. Its benchmark compares agentic and fixed-context review of AI-generated issue-resolution patches.

Yu et al. test auxiliary information and reasoning instructions for security review [20]. In one prompt variant, the model generates supplementary code from the code under review and its commit message. The authors describe cases in which this generated context conflicted with the actual project and caused a known defect to be missed [20, Sec. 3.1.3].

Gloaguen et al. examine repository-level context files for coding agents performing issue-resolution tasks [21]. LLM-generated files did not significantly improve success, although traces showed that agents followed the instructions and performed more actions. Jin and Chen find that elaborate instructions requesting explanations and repairs can increase the rejection of correct implementations in function-level requirement-conformance tasks [22].

2.5.2 Coordination, Feedback, and Output Validation

A multi-agent architecture distributes responsibilities across several agents or roles. Automated-review architectures include both fixed pipelines and adaptive repository exploration [8], [23]. Multiple stages or agents do not in themselves imply autonomous decision-making.

He et al. distinguish central planning with independent task execution from architectures in which planning and execution are both distributed among agents [24, Sec. 2.3.1]. Cai et al. identify complementary patterns: role-based cooperation assigns specific responsibilities, cross-reflection lets agents examine one another's outputs, and an agent evaluator independently assesses other agents' performance, decisions, or outputs [25, Sec. 4.3].

Phoenix provides a concrete example of role separation and feedback in GitHub issue resolution. A central orchestrator coordinates planning, coding, testing, and failure analysis. When testing detects new failures relative to the original repository state, the failure analyst sends diagnostic feedback to the coder for a further attempt, with at most two retry cycles [26, Sec. III].

Validation can also be applied directly to generated review comments. In RovoDev, a factuality judge checks whether candidate comments align with the code change. A separate ModernBERT classifier, described as an actionability check, predicts whether a comment is likely to lead to code resolution [7]. It was fine-tuned on historical RovoDev-generated comments labelled by subsequent resolution. The pipeline separates the criteria used to generate findings from the filters governing which findings reach developers.

The AutoCommenter production service similarly applies confidence thresholds, changed-line filters, and suppression rules [6]. Practices in historical training data became obsolete, while feedback analysis and independent ratings identified technically correct but low-value comments. Suppression rules allowed selected predictions to be disabled without full retraining.

Table I brings together the representation of criteria, their execution, and the control of findings, including the Beko deployment [27] and RepoReviewer architecture [23].

TABLE I

Criteria and Execution in Reviewed Automated Code-Review Systems

System	Review criteria	Selection and execution	Relevance and limitations for this study
LLM-Reviewer [10]	Fixed prompt with eight categories and 14 checks	All checks supplied together for changed code	Fixed-checklist prompting example; educational setting and limited system context reported by participants
AutoCommenter [6]	Coding practices learned from comments linked to practice documents	Model predictions followed by confidence, location, and suppression filters	Explicit practice references and output filtering; no item-by-item checklist execution
iCodeReviewer [9]	38 specialised security prompt experts	Code-feature routing, check-specific context retrieval, analysis prompts, and proposition aggregation	Selection and decomposition of security checks; evaluation on an internal company dataset
RovoDev [7]	Code, test-file, and comment guideline groups	Joint prompting followed by factuality and actionability filters	Prior thematic guidance and filtering; no reported item-level contribution analysis
CodeReviewBot at Beko [27]	Internal criteria not detailed in the case study	Qodo PR-Agent adapted to Azure DevOps alongside human review	Evidence about deployment and user response; insufficient detail for criterion-level replication
RepoReviewer [23]	No concrete checklist published	Model-assisted context synthesis and file analysis; programmatic selection and deduplication; final reporting	Architecture example; effectiveness benchmark remains incomplete

System	Review criteria	Selection and execution	Relevance and limitations for this study
SWE-Review [8]	Issue requirements and repository evidence gathered by the reviewer	Adaptive repository exploration, command execution, and review–revision loop	Repository tools and executable evaluation; broader API quality checks remain outside its measures

Table I distinguishes how criteria are represented, how checks are executed, and how findings are filtered. Comparisons of complete systems therefore combine several design differences and cannot isolate the contribution of checklist content, execution, or context alone. The following section examines the datasets and measures used to evaluate review systems.

2.6 Evaluation Data and Measures

2.6.1 Review Cases and Reference Information

The reviewed studies use different types of evaluation examples, including individual functions, source files, and changes within complete repositories. Table II summarises these datasets, the reference information available for assessing review results, and their suitability for evaluating the proposed system.

TABLE II

Evaluation Data in Code-Review Studies

Source	Evaluation unit and reference	Useful design principle	Main limitation for this study
SWE-Review [8]	1,384 patches for 500 tasks, with issues, repositories, and executable tests used for benchmark evaluation	Include successful and unsuccessful patches; assess decisions and revision outcomes	Issue-resolution correctness does not cover documentation, maintainability, or all other review criteria
Jin and Chen [22]	Correct and defective functions for the same requirement	Evaluate false rejection as well as missed defects	Small functions do not reproduce PR context
iCodeReviewer [9]	682 programs: 345 with confirmed vulnerabilities and 337 benign cases from confirmed false-positive reports	Include negative cases and localisation information	Internal company dataset focused on security issues
Gonçalves et al. [2]	Three changes in the Java text editor jEdit, with known functional defects, including seeded defects	Control task complexity and expected findings	Small number of prepared human-review tasks
Braz et al. [5]	Prepared Java change with two seeded vulnerabilities	Validate controlled defects and task materials	One change and two security concerns
Yu et al. [20]	534 files from human-reviewed OpenStack and Qt changes, each with a known target security defect	Assess type, location, and explanatory value separately	Known-target files do not measure false alarms on correct changes

Prepared tasks make expected problems inspectable but can simplify real development. Braz et al. validated and refined their experimental materials through expert review and pilot work [5].

Historical comments are useful evidence but not a complete inventory of defects. AutoCommenter notes that reviewers can overlook problems, mention recurring issues only once, or prioritise other concerns [6]. Tufano and Bavota also identify limitations of comparing generated comments with human comments by similarity [16]: different wording may describe the same defect, and a valid additional finding may have no historical counterpart. Such findings require independent assessment rather than automatic classification as errors. Executable tests provide another form of reference evidence, as in SWE-Review [8], but establish only the properties they cover.

2.6.2 Detection, Explanation, and Usefulness

Precision, recall, and F1 characterise detection performance in studies such as iCodeReviewer [9]. Their interpretation depends on the evaluation unit and reference coverage: recall describes recovery of known problems, while additional findings require the assessment discussed above.

Yu et al. rate responses using defect type, location, and description [20]. An *Instrumental* response identifies the target precisely and explains it sufficiently. A *Helpful* response contributes to identifying it but is less complete or precise. A *Misleading* response provides no useful information about the target. An *Uncertain* response misses the target and raises another possible issue that assessors cannot confirm. Uncertain therefore describes a limit of the assessors' knowledge, rather than model-expressed uncertainty or a confirmed error.

I-Score is the percentage of Instrumental responses. IH-Score combines the percentages of Instrumental and Helpful responses, whereas M-Score is the percentage of Misleading responses. All three denominators include responses from all four categories [20]. These measures assess responses about known targets. They do not directly measure false alarms on correct changes or specify how to aggregate multiple findings within a PR.

A correct review decision does not guarantee accurate or useful feedback. Jin and Chen show that a correct rejection can be accompanied by an incorrect fault explanation [22]. AutoCommenter reports technically correct suggestions with insufficient practical value [6]. These findings support separate assessments of the review decision, the explanation, and the practical value of a comment.

Industrial studies also assess usefulness through developer responses: AutoCommenter examines feedback and subsequent edits [6], RovoDev measures code resolution in the next commit [7], and Beko uses author-assigned comment statuses [27]. These indicators capture how developers respond to findings; establishing whether a reported problem exists requires a separate assessment.

2.6.3 Resource Costs, Repetition, and Human Cognitive Load

Token consumption and execution time complement quality measures. Gloaguen et al. report that LLM-generated repository context files increased average costs without a statistically significant success-rate gain in their tested issue-resolution settings [21]. Yu et al. exclude cost and latency from their cross-model comparison because invocation modes and deployment environments differ [20]. Cost comparisons therefore need controlled execution conditions as well as a defined review procedure.

Yu et al. ran each model–prompt combination three times and averaged the performance scores to reduce the effect of nondeterminism [20, Sec. 2.3.3]. Repeated runs assess variability, but consistency alone cannot establish correctness: a system may repeatedly miss the same target.

Human cognitive load is a separate construct from computational cost. Krell evaluated StuMMBE-Q in biology education, distinguishing mental load from mental effort [28]. Mental load concerns task demands, while mental effort concerns the cognitive resources a participant reports investing. In their code-review experiment, Gonçalves et al. used two six-item subscales with seven-point responses and averaged item scores to obtain their cognitive-load measure [2, Sec. 4.2.5].

These human ratings must be interpreted alongside review quality [2]. Timing measures also need separation: system runtime measures computation, active review time measures human work, and PR turnaround

additionally includes scheduling and waiting.

2.7 Synthesis and Position of the Proposed Research

The reviewed systems use different forms of review guidance, including fixed checklists, coding-practice rules, and specialised checks selected for the code under review [6]–[10]. They also differ in how they obtain context, perform checks, and validate findings [6]–[9], [23]. Their evaluations cover different tasks and combinations of these mechanisms, making it difficult to isolate the contribution of each design choice.

The proposed system will review backend API changes using thematic checklists for implementation, tests, and documentation. It will select applicable checklists from an initial collection prepared manually from project requirements, API contracts, and technical documentation.

Controlled comparisons will use the same model and review cases. The first comparison will assess review with and without explicit checklists. Subsequent ablations will start from the same checklist set selected for each change and remove only the items or groups under study. Separate comparisons will examine whole-checklist versus stepwise execution and shared versus check-specific context. Findings will be checked against reference evidence to assess correctness and rated for usefulness. Token consumption and system execution time will quantify computational cost.

References

[1] C. Sadowski, E. Söderberg, L. Church, M. Sipko, and A. Bacchelli, “Modern code review: A case study at Google,” in *Proc. 40th Int. Conf. Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2018, pp. 181–190. doi: 10.1145/3183519.3183525.

[2] P. W. Gonçalves, E. Fregnan, T. Baum, K. Schneider, and A. Bacchelli, “Do explicit review strategies improve code review performance? Towards understanding the role of cognitive load,” *Empirical Software Engineering*, vol. 27, Art. no. 99, 2022. doi: 10.1007/s10664-022-10123-8.

[3] V. R. Basili et al., “The empirical investigation of perspective-based reading,” *Empirical Software Engineering*, vol. 1, no. 2, pp. 133–164, 1996. doi: 10.1007/BF00368702.

[4] C. Y. Chong, P. Thongtanunam, and C. Tantithamthavorn, “Assessing the students' understanding and their mistakes in code review checklists: An experience report of 1,791 code review checklist questions from 394 students,” in *Proc. 43rd IEEE/ACM Int. Conf. Software Engineering: Software Engineering Education and Training (ICSE-SEET)*, 2021, pp. 20–29. doi: 10.1109/ICSE-SEET52601.2021.00011.

[5] L. Braz, C. Aeberhard, G. Çalikli, and A. Bacchelli, “Less is more: Supporting developers in vulnerability detection during code review,” in *Proc. 44th IEEE/ACM Int. Conf. Software Engineering (ICSE)*, 2022, pp. 1317–1329. doi: 10.1145/3510003.3511560.

[6] M. Vijayvergiya et al., “AI-assisted assessment of coding practices in modern code review,” in *Proc. 1st ACM Int. Conf. AI-Powered Software (AIware)*, 2024. doi: 10.1145/3664646.3665664.

[7] K. Tantithamthavorn et al., “RovoDev Code Reviewer: A large-scale online evaluation of LLM-based code review automation at Atlassian,” in *Proc. 48th IEEE/ACM Int. Conf. Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2026. doi: 10.1145/3786583.3786851.

[8] R. Wang et al., “SWE-Review: Closing the loop on issue resolution with agentic code review,” arXiv preprint, arXiv:2607.06065, 2026. doi: 10.48550/arXiv.2607.06065.

[9] Y. Peng, K. Kim, L. Meng, and K. Liu, “iCodeReviewer: Improving secure code review with mixture of prompts,” in *Proc. 40th IEEE/ACM Int. Conf. Automated Software Engineering (ASE)*, 2025. doi: 10.1109/ASE63991.2025.00264.

[10] E. Oliveira, M. Fu, P. Thongtanunam, S. López-Pernas, and M. Saqr, “AI-assisted code review as a scaffold for code quality and self-regulated learning: An experience report,” in *Proc. 48th IEEE/ACM Int. Conf. Software Engineering: Software Engineering Education and Training (ICSE-SEET)*, 2026. doi: 10.1145/3786580.3786956.

[11] M. E. Fagan, “Design and code inspections to reduce errors in program development,” *IBM Systems Journal*, vol. 15, no. 3, pp. 182–211, 1976. doi: 10.1147/sj.153.0182.

[12] A. Aurum, H. Petersson, and C. Wohlin, “State-of-the-art: Software inspections after 25 years,” *Software Testing, Verification and Reliability*, vol. 12, no. 3, pp. 133–154, 2002. doi: 10.1002/stvr.243.

[13] R. Tufano, “Code review is a conversation: Toward conversational AI review assistants,” arXiv preprint, arXiv:2607.22095v1, 2026. doi: 10.48550/arXiv.2607.22095.

[14] N. Rao, B. Vasilescu, and R. Holmes, “From overload to insight: Bridging code search and code review with LLMs,” in *Proc. 33rd ACM Int. Conf. Foundations of Software Engineering, Companion Proceedings (FSE Companion)*, 2025. doi: 10.1145/3696630.3728518.

[15] N. R. Malotaux, owner, *Inspection Manual: Procedures, Rules, Checklists and Other Texts for Use in Inspections*, ver. 0.45, N R Malotaux – Consultancy, Apr. 15, 2008.

[16] R. Tufano and G. Bavota, “Automating code review: A systematic literature review,” arXiv preprint, arXiv:2503.09510v1, 2025. doi: 10.48550/arXiv.2503.09510.

[17] J. Cohen, S. Teleki, and E. Brown, *Best Kept Secrets of Peer Code Review*. SmartBear Software, 2006, ch. “Questions for a Review Process,” pp. 103–114.

[18] J. Cohen, “Prepare to succeed: A guide to effective code review,” *Better Software*, pp. 30–35, Nov./Dec. 2009.

[19] A. Aggarwal and N. F. Ghalaty, “Self-improving AI coding agents through accumulated behavioral rules: A closed-loop framework,” arXiv preprint, arXiv:2607.13091v1, 2026. doi: 10.48550/arXiv.2607.13091.

[20] J. Yu et al., “An insight into security code review with LLMs: Capabilities, obstacles, and influential factors,” arXiv preprint, arXiv:2401.16310v6, 2026. doi: 10.48550/arXiv.2401.16310.

[21] T. Gloaguen, N. Mündler, M. Müller, V. Raychev, and M. Vechev, “Evaluating AGENTS.md: Are repository-level context files helpful for coding agents?” arXiv preprint, arXiv:2602.11988v2, 2026. doi: 10.48550/arXiv.2602.11988.

[22] H. Jin and H. Chen, “Are LLMs reliable code reviewers? Systematic overcorrection in requirement conformance judgement,” *Automated Software Engineering*, vol. 33, Art. no. 90, 2026. doi: 10.1007/s10515-026-00638-5.

[23] P. Zhang, “RepoReviewer: A local-first multi-agent architecture for repository-level code review,” arXiv preprint, arXiv:2603.16107, 2026. doi: 10.48550/arXiv.2603.16107.

[24] J. He, C. Treude, and D. Lo, “LLM-based multi-agent systems for software engineering: Literature review, vision, and the road ahead,” *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 5, Art. no. 124, pp. 1–30, 2025. doi: 10.1145/3712003.

[25] Y. Cai, R. Li, P. Liang, M. Shahin, and Z. Li, “Designing LLM-based multi-agent systems for software engineering tasks: Quality attributes, design patterns and rationale,” arXiv preprint, arXiv:2511.08475v1, 2025. doi: 10.48550/arXiv.2511.08475.

[26] K. Koech, M. Adam, B. B. Jean Jacques, and J. Barros, “Phoenix: Safe GitHub issue resolution via multi-agent LLMs,” arXiv preprint, arXiv:2606.20243v2, 2026. doi: 10.48550/arXiv.2606.20243.

[27] U. Cihan et al., “Automated code review in practice,” in *Proc. 47th IEEE/ACM Int. Conf. Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2025, pp. 425–436. doi: 10.1109/ICSE-SEIP66354.2025.00043.

[28] M. Krell, “Evaluating an instrument to measure mental load and mental effort considering different sources of validity evidence,” *Cogent Education*, vol. 4, no. 1, Art. no. 1280256, 2017. doi: 10.1080/2331186X.2017.1280256.